Deadline-Aware Food Delivery: Problem Template

Updated 7 November 2026

Purpose

This is the one problem everyone in the group solves. Everyone uses the same data, the same parameters and the same scoring code, so all results can be compared.

Deliver as many orders on time as possible, then minimise the average wait of delivered orders.

Scenario

One rider picks up several orders at one restaurant and delivers all of them in one trip. Each dish stays fresh for a different time: ice cream lasts 30 minutes, rice lasts 90. The customer who ordered first is not always the one whose food spoils first.

Classic TSP assumption	Our use case
Every city is equally important	Each order has its own deadline
Only the visit order matters	The arrival time at each stop matters
One goal: shortest distance	Goals: most orders on time, then shortest average wait
Must visit every city	Orders that can't be on time are cancelled
The tour returns to the start	The route ends at the last customer

Problem definition

Input (all known at the start; no new orders arrive later)

- One restaurant r and n customers
- Travel time w(i, j) between any two places, in minutes
- For each customer k: order time o_k, food type, shelf life ℓ_k
- The rider leaves at t = 0, the time of the last order. No cooking time.

Output

- The order in which the rider visits customers, starting at r. Customers not in the route are cancelled.

Rules

- 1 rider, carries all orders, does not return to the restaurant
- Same travel time both ways, no traffic, no time spent at each door
- Wait of customer k = arrival time a_k − order time o_k

- Every delivered order must arrive by its deadline d_k:

Rule	Deadline d_k	Meaning
Food freshness	$o_k + \ell_k$	Food must not spoil

Goals (in this order)

1. Deliver as many orders as possible (cancel as few as possible)
2. Among those, lowest average wait of delivered customers

We always also report the longest wait and the number of cancelled orders.

Two useful facts about this problem:

- **Deadline = wait limit.** $a_k \le o_k + \ell_k$ is the same as "wait ≤ shelf life".
- Waiting time depends on which customers are delivered. Total wait is the sum of arrival time minus order time over delivered customers. For a fixed delivered count, minimising total wait minimises average wait. Replacing wait with arrival time is valid only when the delivered customer set is unchanged.

Fixed parameters and data

These parameters are defined in Common.java. Keep them shared across all methods so that the comparisons use the same data and assumptions.

Parameter	Value	Why
Restaurant	Bukit Bintang Plaza (GeoNameID 7284944, class S / MALL)	Busy food area in central KL
Customer locations	Populated places (class P) within 6 km of the restaurant: 33 places	A compact urban delivery area with varied distances
Travel time	Straight-line distance ÷ 20 km/h	Simple; the low speed covers detours, traffic lights and jams
Food types	Ice cream 30 min, noodles 60 min, rice 90 min; each order picks one at random	Different shelf lives create the conflict: who ordered first is not who is most urgent
Order times	Random within the 10 min before the rider leaves; the last order is at t = 0	All orders are known at the start

Data used from MY.txt: columns GeoNameID, name, latitude, longitude and feature class. Duplicate coordinates are dropped.

Data we add: order time, food type and shelf life for each order. These are stored in the instance files, not in MY.txt, and are linked by GeoNameID. MY.txt itself is not edited.

The 33 nearby locations keep the delivery scenario geographically compact. Small batches allow comparison against the exact solver; larger batches test speed and overload. Record selection and limitations are explained in docs/data_decisions.md.

Experiment variables

Vary the order count and shelf-life multiplier. Use seeds 1–30 for each setting and report the mean and spread: 6 order counts × 3 multipliers × 30 seeds = 540 inputs.

Variable	Flag	Values used	What it tests
Number of orders n	`--n`	5, 8, 10 (small); 20, 50, 100 (large)	Small: compare with the exact best. Large: speed, since brute force can't finish (10! ≈ 3.6 million orders, 15! ≈ 1.3 trillion)
Shelf-life tightness	`--tight`	0.5, 1.0, 1.5	Multiplies every shelf life; tighter means more conflict

When n is larger than 33, places are reused (several customers in the same neighbourhood). With one rider, large n will cancel many orders; those runs are mainly for speed.

Files and shared tools

The algorithms and experiment tools use Java 17+ with no third-party libraries. Python 3.10+ with matplotlib is needed only to regenerate charts. Shared Java tools are in src/com/delivery/tools/.

File	What it does
Common.java	The fixed parameters above, plus travel time and deadline helpers
GenerateInstance.java	Makes one instance from MY.txt: `--n`, `--seed`, `--tight`
Evaluate.java	Scores a route: delivered, cancelled, late, average wait and longest wait
BruteForce.java	Exact reference for small inputs; used for n ≤ 10
NearestNeighbor.java	Baseline: classic nearest neighbour, then drops late orders

An instance file (JSON) holds the restaurant, the customers and the full travel-time matrix:

```
{"n": 6, "seed": 10, "tight": 1.0,
 "restaurant": {"geonameid": "7284944", ...},
 "customers": [{"id": 1, "order_time": -3.5,
                "food": "ice_cream", "shelf_life": 30.0, ...}, ...],
 "travel_min": [[0, 2.63, ...], ...]}
Matrix index 0 is the restaurant; index k is customer k.
```

Implement a solver returning customer IDs in delivery order. Instance.toSolverInstance() adapts shared input to the solver model; RunExperiments converts solver IDs 0..n−1 back to instance IDs 1..n. Evaluate scores every route independently. A valid solution always has late = 0.

Windows: .\run_tests.bat
macOS/Linux: bash run_tests.sh
Add --all to either command to generate all 540 inputs, run the comparison and draw charts. Run from the project root. See README.md for setup.

Worked example (n = 6, seed 10)

The exact best route delivers all six orders, with an average wait of 29.0 minutes. Sentul waits 62.7 minutes, within its rice shelf life of 90 minutes. Nearest neighbour cancels one ice cream order.

File: data/processed/example_n6_s10.json. The rider leaves Bukit Bintang Plaza at t = 0. This fixed example is reconstructed from the table rather than generated by the seeded Java sequence.

Id	Customer	Ordered (min)	Food	Shelf life	From restaurant (min)
1	Golden Triangle	−3.5	Ice cream	30	2.6
2	Kampong Baharu	−3.2	Ice cream	30	6.9
3	Sentul	−5.3	Rice	90	14.8
4	Pudu	0.0	Noodles	60	5.3
5	Kuala Lumpur Village	−7.8	Ice cream	30	8.5
6	Taman Maluri	−3.2	Noodles	60	8.0

Solution	Route	Cancelled	Average wait (min)	Longest wait (min)
Exact best	1 → 2 → 5 → 4 → 6 → 3	0	29.0	62.7 (Sentul)
Nearest neighbour	1 → 2 → 4 → 6 → 3	1 (Kuala Lumpur Village)	21.9	49.8

What this shows:

- **Nearest neighbour** goes to Pudu and Taman Maluri before the far ice cream at Kuala Lumpur Village, which then spoils. Its average wait looks lower only because it served one fewer customer. That is why the goals are compared in order: cancelled first, wait second.
- The exact best route serves the three ice creams first and leaves the rice at Sentul for last, since rice lasts longest.

Rules for the group

Every method must:

1. Use the shared GenerateInstance.java inputs and Evaluate.java scoring. Keep all fixed parameters in Common.java; do not create private copies.
2. Return only valid routes (`late = 0`).
3. Use the same 540 input files for both redesigned approaches and the reference methods.
4. Report for each setting: delivered, cancelled, average wait, longest wait, runtime and, for $n \leq 10$, the gap to BruteForce.java.
5. Compare results in goal order: fewer cancelled wins; average wait only breaks ties.

What counts as a new approach:

- **Counts:** a different algorithm idea (for example step-by-step greedy vs build a full route and then improve it) or a different main data structure.
- Different parameter settings of the same implementation do not count as distinct approaches. Order count and shelf-life multiplier are experiment settings, not separate algorithms.

The two implemented approaches

- Approach A: SolverA.java starts with deadline order, then repeatedly improves the full plan by moving one stop at a time.
- Approach B: SolverB.java chooses one stop at a time using feasibility, orders lost, travel time, slack and customer ID.